Camera Streaming Pipeline/Camera Stream over WIFI

How to Run:
Run server/stream:
gcc nodelayrtsp.c $(pkg-config --cflags --libs gstreamer-rtsp-server-1.0)
./a.out

Connect to stream as client (new window for each connection):
gst-launch-1.0 rtspsrc location=rtsp://localhost:8554/front latency=0 drop-on-latency=true ! rtph265depay ! queue ! h265parse ! avdec_h265 ! videoconvert ! ximagesink

gst-launch-1.0 rtspsrc location=rtsp://localhost:8554/back latency=0 drop-on-latency=true ! rtph265depay ! queue ! h265parse ! avdec_h265 ! videoconvert ! ximagesink

Installed:
Main gstreamer install for linux ubuntu/debian:
apt-get install libgstreamer1.0-dev libgstreamer-plugins-base1.0-dev libgstreamer-plugins-bad1.0-dev gstreamer1.0-plugins-base gstreamer1.0-plugins-good gstreamer1.0-plugins-bad gstreamer1.0-plugins-ugly gstreamer1.0-libav gstreamer1.0-tools gstreamer1.0-x gstreamer1.0-alsa gstreamer1.0-gl gstreamer1.0-gtk3 gstreamer1.0-qt5 gstreamer1.0-pulseaudio

For rtsp camera stream:
sudo apt install libgstrtspserver-1.0-dev
sudo apt install ffmpeg (for rtsp stream)

How to find sink on jetson:

⭐ Method 1 — Run with GST_DEBUG and watch what autovideosink picks

This is the most direct and accurate way.

Run your pipeline like this:

Bash Copy

```bash
GST_DEBUG=2 gst-launch-1.0 videotestsrc ! autovideosink
```

Or for your RTSP client:

Bash Copy

```bash
GST_DEBUG=2 gst-launch-1.0 rtspsrc location=rtsp://... ! \
  rtph264depay ! avdec_h264 ! videoconvert ! autovideosink
```

Then look for lines like:

Code Copy

```code
autovideosink: using xvimagesink
```

or

Or can use this command: gst-inspect-1.0 autovideosink
Can even test with the different possible sinks and see which gives lowest latency

Source - ribbon camera: nvarguscamerasrc
Source - usb camera: v4l2
Rover cameras are usb, mjpeg and yuy, format image/jpeg and video/x-raw, NOT NVMM like csi ribbon camera.

Used these mixed with stuff from the nvidia documentation + internet

✅ A correct full pipeline (MJPEG → NVENC)

Code Copy

```
v4l2src device=/dev/video2 ! \
image/jpeg,framerate=30/1 ! \
jpegdec ! \
nvvidconv ! \
'video/x-raw(memory:NVMM),format=NV12' ! \
nvv4l2h264enc bitrate=4000000 ! \
h264parse ! \
rtph264pay ! udpsink host=... port=...
```

This is the standard pattern for Jetson + USB camera + GPU encoding.

Yuy pipeline

🥇 Lowest-latency pipeline (USB camera → RTSP → GPU encode)

1. Use YUYV instead of MJPEG (if your camera supports it)

MJPEG requires CPU decode → adds ~10–20 ms.

YUYV is raw → no decode → lower latency.

Caps:

Code Copy

```
v4l2src device=/dev/video2 io-mode=1 ! \
video/x-raw,format=YUY2,width=1280,height=720,framerate=30/1 ! \
nvvidconv ! \
'video/x-raw(memory:NVMM),format=NV12' ! \
nvv4l2h264enc insert-sps-pps=true iframeinterval=15 bitrate=4000000 preset-level=1 !
h264parse ! \
rtph264pay config-interval=1 name=pay0 pt=96
```

Client connection line - this if client has gpu - probably don't need:
gst-launch-1.0 rtspsrc location=rtsp://localhost:8554/front latency=0 drop-on-latency=true ! rtph265depay ! queue ! h265parse ! nvv4l2decoder ! nv3dsink -e

Old way to connect - what caused most of the delay
ffplay rtsp://localhost:8554/front

GStreamer Documentation + Tutorials:
https://gstreamer.freedesktop.org/documentation/tutorials/basic/concepts.html?gi-language=c
Basic Gstreamer Overview (cmd): https://www.youtube.com/watch?v=tv1Mm7q_GIE
Understanding General Pipeline: https://www.youtube.com/watch?v=bFS8SCZbX14
RTSP Gstreamer Video: https://www.youtube.com/watch?v=2Y77adCwycw&t=607s
Rtspsrc Gstreamer for client connect:
https://gstreamer.freedesktop.org/documentation/rtsp/rtspsrc.html?gi-language=c
Gstreamer + Nvidia Documentation:
https://docs.nvidia.com/jetson/archives/r34.1/DeveloperGuide/text/SD/Multimedia/AcceleratedGstreamer.html
Other Source for GPU Pipeline:
https://developer.ridgerun.com/wiki/index.php/Jetson_Nano/Gstreamer/Example_Pipelines/Transforming